How Much Do Co-Located Containers Really Share? A Pairwise Measurement Study

Hanlin Wu
KU Leuven
Leuven, Belgium
hanlin.wu@student.kuleuven.be

Eddy Truyen
KU Leuven, DistriNet
Leuven, Belgium
eddy.truyen@kuleuven.be

Wouter Joosen
KU Leuven, DistriNet
Leuven, Belgium
wouter.joosen@kuleuven.be

Abstract

This paper investigates for contemporary container platforms how much memory saving can be attained by co-locating instances of one image on the same node – as a FaaS platform does to keep a warm pool ready, or as a blue-green deployment does across consecutive versions, pinning different tenants to the version they were validated against. The expectation is that images built from overlapping layers let the kernel back them with a single physical copy. That expectation has never been verified for a specific pair of instances, and standard metrics cannot verify it: Proportional Set Size (PSS) divides a page's cost by its mapcount, and so reports sharing with *any* process on the host rather than with the neighbour a scheduler is reasoning about. We measure it directly, intersecting the resident page frames of two named containers through the kernel's `pagemap` interface. Between two consecutive builds of a multi-tenant Java-based application, whose application layers are byte-identical, the instances share 2.4 MiB of mapped memory – under half a percent of either one's resident footprint. A static analysis of roughly 100 releases each of two widely-used Java applications finds that most consecutive release pairs share no image snapshot whatsoever. The cause is structural rather than semantic. A container runtime names each unpacked layer by the hash chain of every layer beneath it, so a metadata-only difference early in the stack gives byte-identical files above it fresh inodes, and the kernel cannot deduplicate them. On current mainstream container platforms, a sharing discount should not be assumed from image-layer similarity alone, particularly once co-located versions are built independently. Existing proposals that key image storage on content rather than layer position deserve consideration as a way out.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; • **Software and its engineering** → *Software deployment*; *Virtual machines.*



This work is licensed under a Creative Commons Attribution 4.0 International License.

WoAIS '26, Tarragona, Spain

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-3108-2/2026/12
https://doi.org/10.1145/3861043.3861046

Keywords: container images, memory sharing, page cache, cgroups, multi-tenancy, serverless cost-effectiveness, Kubernetes, JVM

ACM Reference Format:
Hanlin Wu, Eddy Truyen, and Wouter Joosen. 2026. How Much Do Co-Located Containers Really Share? A Pairwise Measurement Study. In *Workshop on AI and Serverless (WoAIS '26), December 14–18, 2026, Tarragona, Spain.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3861043.3861046

1 Introduction

Container-based platforms pack many containers onto one host to improve resource utilization. FaaS platforms keep pools of warm containers ready to absorb invocations without a cold-start penalty, and co-locating instances of the same image reduces cold starts further [1]. Blue-green deployments similarly keep several versions of one service live, pinning each tenant to the version they were validated against rather than forcing everyone onto whatever is current; a placement policy can co-locate different versions of one image on a node specifically to maximize this resource sharing [14].

A recurring proposal for increasing memory utilization is to credit the memory co-located instances are expected to share: if two containers are built from overlapping image layers, the kernel's page cache can map the same physical pages into both address spaces instead of duplicating them, lowering an added instance's marginal cost below its declared footprint. Bravo Ferreira et al. state this explicitly for FaaS workloads and propose a scheduler that maximizes this assumed overlap [2]; Truyen et al. likewise build it into a cost model for co-located versions, treating it as substantial [14].

Both rely, however, on the `/proc/<pid>/smaps` interface or PSS, which marks a page *Shared* whenever more than one process on the host maps it. This is real, kernel-verified information, but not *attributable*: it says a page is shared with some other process, not which one. On a host running only the pair of interest this does not matter; it does on a real cluster also hosting a database, sidecars, and unrelated workloads.

Attribution is only half the problem: crediting shared layers also assumes they resolve to common physical memory,

which requires both containers to map the same inode. Container runtimes name each unpacked layer by the hash chain beneath it, so a difference anywhere earlier in the chain – even metadata only – gives every later layer a fresh inode, faulting identical files in twice. How often this happens across a real application's releases, and how much sharing it removes, has not been measured.

In this paper, we introduce a simple measurement technique that intersects the resident physical page frames of two *named* containers directly, via `/proc/<pid>/pagemap`, giving a pairwise, ground-truth answer to how much two specific instances share right now, independent of anything else running on the host. We apply it to a real, evolving JVM-based service to measure how much two instances of the identical image share while running, and how much of that survives once the images differ – as between two tenant-pinned versions of one service – complemented by a static characterization of image-layer overlap across many consecutive versions of two applications.

2 Problem Statement

When a platform keeps several versions of one container image resident on a node – during a blue-green migration, or to hold tenants on the version they were validated against – the question that decides capacity is whether the second instance costs a full memory footprint or a discounted one. Existing cost and placement models answer with a per-pair term: the fraction f of libraries two arbitrary containers are assumed to share [2], or the *Shared* quantity by which two co-located versions of one service are discounted [14]. Both are properties of a *specific pair of instances*, and both are consumed as model input rather than measured. Establishing what that term is actually worth is the problem this paper addresses.

The quantity has a precise definition. Linux shares identical content between processes through its virtual-memory subsystem: each address space is a set of virtual memory areas (VMAs) mapping anonymous regions or file contents, and the frames backing a file-backed VMA come from the kernel's file cache, indexed by the file's inode [8]. Two processes share a *file-backed* page's frame only when both map content resolving to the same inode [6]; bytes read from a separate copy, at a different path and hence a different inode, are faulted in twice. Residency in both processes is necessary but not sufficient. A shared library is mapped `MAP_PRIVATE`, so a write by either side quietly replaces that side's copy of the page and leaves the other mapping the original – the page stays resident in both, but is no longer one frame. What two instances share is therefore the set of physical frames they *both* hold resident, and that is what we measure. Content a process reads without mapping falls outside the definition, since the page cache deduplicates it when the inode matches but it never enters a page table; our figures accordingly bound mapped sharing rather than all sharing (§6).

No existing interface reports it. RSS counts every page a process maps, whatever else maps it. PSS divides each page's cost by its mapcount [4], and `smaps` splits resident memory into *Shared* (mapcount > 1 anywhere on the host) and *Private* bytes. All three capture the *degree* of sharing; none captures its *identity*, because mapcount records how many processes map a page and never which ones. On a machine running only the pair of interest the distinction collapses, unremarked. On a production node also running a database, sidecars and unrelated tenants, an instance's Shared bytes include pages shared with none of the peers under consideration, so they overstate what moving a peer away would reclaim. Nor does measuring whole hosts recover it: one configuration's realized saving does not decompose into the per-pair quantities a cost model needs, and configurations are exponential in instance count. What is needed is the intersection itself – the physical page-frame numbers (PFNs) both instances hold resident – which §4.1 computes directly.

A cheaper proxy might seem possible: `/proc/<pid>/maps` prints a device and inode per mapping. But those columns are unreliable – on an overlay mount we observed three containers report three different device numbers for one file – and inode equality is necessary, not sufficient: it says nothing about how many pages were faulted in, nor does a shared inode survive one side writing to the page. Only resident page frames settle how much two instances actually share.

3 Related Work

Bravo Ferreira et al. [2] first demonstrated that shared image layers can produce genuine physical sharing, showing via `pmap` a kernel-tracked *Shared* byte count for two containers union-mounted over a common library layer, none when the same content sits in a private file system. They build a bin-packing model around a free parameter f, the fraction of libraries two containers are assumed to share, and show even a small f yields most of the achievable packing-density gain, motivated explicitly by FaaS platforms – but f is never measured for a real application. Truyen et al. [14] apply the same mechanism to two *consecutive versions* of one application resident on a node during a migration step, reproducing that `smaps`-based measurement to support a cost model where demand is proportional to co-located instances, discounted by assumed sharing.

Cheng et al. [3] take layer-wise sharing as the basis for a placement algorithm rather than a cost model. Measuring host buffer/cache footprint as containers are spawned together or spread across machines, they find it stays flat rather than growing with instance count, and build heuristics that co-locate workflow functions to exploit that effect. Their quantity, measured at the host level, includes content a

process reads without mapping, complementary to ours (§6). The workflows they place share a common runtime layer at the bottom of each image, exactly where §5.1 shows the shared-layer condition holds. Our results say what happens when that prefix is not intact, as when images are built independently, suggesting the benefit such an algorithm assumes is more fragile than a single shared base image makes it look.

Medes [10] also attributes redundancy to a specific pair of sandboxes, but differently: it fingerprints memory chunks by content hash and merges matches regardless of whether the kernel's inode-based cache would share them. Across this line of work, sharing is confirmed possible, engineered, or built into a cost model, but never measured for a real, evolving application, nor tied to the mechanism that grants or withholds it. This paper measures that quantity – the fraction Bravo Ferreira et al.'s model calls f, and Truyen et al.'s *Shared* – directly and attributably, for both co-location patterns a scheduler must reason about.

4 Measurement Methodology

4.1 Pairwise PFN-Intersection Tool

To measure realized sharing between any two specific, named containers, we built a tool that, for each container, walks `/proc/<pid>/maps` to enumerate its virtual memory areas and reads `/proc/<pid>/pagemap` [4] to resolve every resident virtual page to the PFN backing it. Given two containers, the tool computes the intersection of their resident PFN sets: any PFN present in both sets is a physical frame genuinely mapped by both containers at the moment of measurement, independent of whether the kernel's own `smaps` bookkeeping happens to also attribute that page to some other, unrelated mapper elsewhere on the host. It reports the intersection broken down by mapped file, so sharing can be attributed to the specific library or archive responsible, and separately intersects each container's *anonymous* memory, non-zero only if same-page merging (KSM) is active – ruling KSM in or out as a confound rather than assuming it away. Reading `pagemap`'s physical frame numbers requires elevated privileges, so the tool runs on the node, not inside either measured container; we use it as an offline, ground-truth instrument for testing cost-model assumptions, not as a mechanism intended to run inside a live scheduler. Applying it is independent of application or runtime – only §4.2 below is Keycloak-specific. The tool, the release-tag overlap script, and the raw measurements behind our figures are available at https://github.com/k8-scalar/memory-sharing-between-container-pairs.

4.2 Application and Workload

We study Keycloak, an open-source identity and access management (IAM) application, for three reasons: it is JVM-based, so the shared artifacts central to §2 are present and measurable; it is natively multi-tenant, its *Realm* construct isolating each tenant in one instance, exactly what a blue-green migration must reason about; and it has an actively maintained history spanning dozens of versions, which §5.1 needs to test the mechanism across real, independently built releases. Keycloak is not the highest-stakes case – §6 argues that is serverless model-serving, where mapped weights dwarf a JVM's artifacts – which makes it a conservative choice: if the mechanism collapses sharing here, it is unlikely to spare an application with more at stake. We drive it with a fixed per-tenant workload: a login request, several backend introspection requests, a token refresh, more introspection requests, and a logout, repeated by each simulated user for the duration of a run, with randomized think-time between requests. Fixing this behavior keeps memory measurements comparable across runs and versions. We report results from a Kubernetes-hosted deployment of this workload, not from a commercial FaaS platform's own execution environment (cfr. threats to validity in §7).

4.3 Deployment Topology

Experiments ran on a three-node Kubernetes cluster (one control-plane, two worker nodes), with all Keycloak instances and their PostgreSQL database on one worker node and load-generation and monitoring on the other. Workload was generated with K6 [5], one pod per simulated tenant running the request sequence described above. Pod-level CPU and memory metrics were collected every 15 s from each node's `/proc` and `/sys` hierarchies.

5 Quantifying Memory Sharing

5.1 Static Layer Overlap

Whether two instances can share a mapped file at all is settled before either runs, by how their images were built. Images are stacked as layers, and when a container starts the runtime decompresses each layer once into its own *snapshot* directory and stitches the relevant snapshots together through a union (overlay) file system [9, 13]. Overlay mounts route a mapped file through to the underlying lower-layer file, so the inode backing a mapped library is the one inside that snapshot directory. Crucially, matching content is not enough to put two images in the same snapshot: the runtime names each snapshot by its *ChainID*, the cumulative hash of the uncompressed digests (diffIDs, from the image configuration rather than the manifest) of every layer at or below that position [9]. A shared snapshot at position k therefore requires layers $0, \ldots, k$ to be identical. If any earlier layer differs at all, every later layer is unpacked into a fresh, separately-inoded snapshot regardless of its own content, so two images can share every byte of an application's binaries and still map them through different inodes. A copy-up reaches the same outcome more simply: a container that writes to a file, or merely changes its mode at entrypoint, gets a private copy with its own inode even when the ChainIDs match. Both are

Table 1. Static snapshot overlap by version-step granularity: mean and max snapshot overlap as a percentage of image bytes, and the number of consecutive-release pairs *n* each figure is computed over, and *zero*, the number of those pairs sharing no snapshot at all. The build granularity applies only to Keycloak.

	Build	Patch	Minor	Major
Keycloak (v20–v26, 96 tags)				
mean	1.15%	1.34%	0.00%	0.00%
max	13.1%	13.1%	0.0%	0.0%
n	19	64	6	6
zero	15	38	6	6
Elasticsearch (v8–v9, 121 tags)				
mean	—	2.20%	0.87%	0.00%
max	—	5.7%	5.6%	0.0%
n	—	97	22	1
zero	—	49	18	1

properties of per-layer snapshotters such as overlayfs under containerd and Docker's `overlay2`; content-addressed designs keying storage on file content instead, such as composefs [7], can share identical files across different layer chains.

This gives a static bound we can compute without running anything. Comparing the ChainIDs of two images from position 0 finds the first position at which they diverge, and every layer before it would share a snapshot were the two ever unpacked on one node. We define *snapshot overlap* between two images as the fraction of total image bytes, weighted by the manifest's per-layer sizes, falling inside that shared prefix: an upper bound, in advance of any runtime measurement, on how much two images could possibly share.

To see how snapshot overlap depends on the size of an upgrade step, we bucket consecutive-release pairs by the level of the semantic-version identifier that changed between them. A *build* step re-packages the same (major, minor, patch) triple with a new build number (e.g. `26.1.2-1` → `26.1.2-2`); a *patch* step changes the last version component (`26.1.2` → `26.1.3`); a *minor* step changes the middle component (`26.1` → `26.2`); and a *major* step changes the first (`25` → `26`). Within each granularity we take one representative tag per version unit – the newest build of each patch, the newest patch of each minor, and so on – and compute snapshot overlap over every consecutive forward pair of those representatives. We collected roughly 100 recent release tags each of two widely-used JVM applications, Keycloak (`v20.x.x-x` through `v26.x.x-x`, 96 tags) and Elasticsearch (`v8.x.x` through `v9.x.x`, 121 tags); the build granularity applies only to Keycloak, whose official image tags carry an explicit build suffix. Table 1 reports the result.

The result is striking, and the means understate it: most consecutive release pairs share no snapshot whatsoever (15 of 19 Keycloak build pairs, 38 of 64 patch pairs, 49 of 97 Elasticsearch patch pairs, and every Keycloak minor and major pair). The non-zero means come from a minority retaining a small base-OS prefix, not sharing spread evenly across the history. This bounds, ahead of any runtime measurement, how much two consecutive releases could possibly share – including the multi-version tenancy case, where co-located versions are typically at least a patch apart: runtime sharing below can therefore never exceed a few percent once images are more than one patch apart, however favorable the runtime conditions.

5.2 Runtime Sharing: Same-Image Case

We co-located two instances of the identical Keycloak image on one node across ten recent patch versions (v26.0.0 to v26.2.2), let each pair settle for 30 s, and measured PFN intersection between them. Realized sharing is small and stable across versions: mean 25.8 MiB (s.d. 0.21 MiB), around 3% of the two-instance RSS union (RSS and PSS both near 900 MiB) and around 5% of one instance's own RSS. We report against RSS because RSS, not PSS, is what a cgroup memory limit enforces (§6); PSS's discount is a host-wide average over however many processes map a page, not an amount attributable to one co-located instance – exactly the ambiguity our pairwise technique replaces. Here the choice barely matters: sharing is small enough that the RSS-sum and PSS-sum land within a percent of each other.

Broken down by mapped file, shared bytes concentrate in a handful of JVM-internal artifacts rather than application code, which matters for how far the result generalizes: `libjvm.so` contributes roughly 16 MiB (92% of its own 17.4 MiB footprint, by far the largest share), `classes.jsa` ~2.7 MiB of 13.6 MiB, and the runtime module image ~3.2 MiB of 148 MiB. Keycloak's own ~144 MiB of application JARs contribute *nothing* to this figure – not because they fail to share, but because the class loader `pread`s rather than maps them, so they never acquire a VMA to intersect. They are shared nonetheless: `fincore` over the containers' snapshot directories shows the 316 JARs under `lib/main` resident in the page cache and held once for both instances, some 122 MiB invisible to a page-frame intersection. The 25.8 MiB of mapped sharing is thus a lower bound. Anonymous-page PFN intersection – non-zero only if same-page merging (KSM) were active – was negligible on our nodes, ruling out KSM: the sharing we observe is entirely file-cache sharing of mapped files, not kernel deduplication of Keycloak's own heap.

5.3 Runtime Sharing: Multi-Version Tenancy Case

If realized sharing survives the smallest possible image change, it is worth modeling in blue-green deployments. To find out, we compared two consecutive *builds* of the same Keycloak patch version, `v26.1.2-1` and `v26.1.2-2`, which bundle the

same OpenJDK build and are about as close to identical as two independently tagged, independently built releases ever get. We measured three pairs under a synchronized 400-VU load ramp: two same-build baselines, as a sanity-check control against §5.2 rather than a result in their own right (both instances at `-1`: 25.84 MiB shared; both at `-2`: 25.86 MiB), and the cross-build pair, at 3% static overlap (Table 1's build column): 2.38 MiB shared. The controls reproduce the same-image finding of §5.2 almost exactly – two independent measurements of the identical configuration agree to within 0.02 MiB. The cross-build pair is the informative one. Both images bundle the same OpenJDK build, so the two processes' `/proc/<pid>/maps` list the same 41 file paths – there is no sense in which the cross-build pair maps *less* of the same content than the same-build pairs do. Yet realized sharing drops by an order of magnitude, from around 26 MiB to 2.4 MiB, and the 2.4 MiB that remains comes entirely from base-OS files (`libc.so.6`, `libm.so.6`, `ld-linux`, locale data) in the layer prefix the two images do still share.

To see exactly what changed, we unpacked all five image layers of both builds and diffed every tar member. The two base layers (L0, L1) are identical. Only seven files, 14 MiB in total, differ in *content*, and all seven sit in L2, the distribution rootfs, alongside 936 members differing only in `mtime`. Above it, L3 – the layer that installs Keycloak itself, including every file whose sharing we measured – has all 444 of its regular files bit-for-bit identical between the builds, and records just six `mtime` differences, all on directories; L4 records three. Since those timestamps sit inside the archives, the uncompressed digests differ wherever the archives do, and the ChainID diverges from L2 upward.

These few timestamp bytes are enough to break sharing across the whole image: once L2's contents differ, L3 and L4 are unpacked into fresh, separately-inoded snapshots *regardless of their own content*. We confirmed this directly rather than inferring it: resolving each container's overlay lowerdir chain and calling `stat` on the resulting snapshot paths puts the two builds' copies of `libjvm.so` at distinct inodes, versus one shared inode for two instances of the same build, the files being byte-identical at 27,258,184 bytes either way. The unmapped content divides the same way: the two builds' JAR sets resolve to different inodes, turning the 122 MiB cached once for a same-image pair into two copies (122 and 131 MiB). A manifest still showing large declared overlap – L0's 22 MiB base layer untouched, L3's 156 MiB Keycloak-install layer byte-identical – thus yields almost none of the physical sharing it suggests, purely because of where in the stack the six-byte difference sits.

6 Implications for Cost-effective Scheduling Policies

These results argue against crediting assumed sharing in placement or capacity decisions. Even the most favorable case maps only a few percent of either instance's resident memory in common, and that ceiling collapses by an order of magnitude once co-located versions diverge – which they routinely do once a platform serves tenants from independently released versions. A scheduler built on an overlap-aware placement model [2] would be optimizing for a benefit largely absent in exactly the scenario it targets. The collapse is partly a build artifact: pinning layer timestamps would recover the gap from 2.4 MiB to 25.8 MiB, reaching but not passing the same-image ceiling, a property of the application rather than the store. Keying snapshot identity on *content* rather than layer *position* – composefs [7] being the most developed proposal – addresses the cause directly without demanding reproducible builds from every producer.

Structurally, crediting sharing inside an instance's declared memory limit is unsafe however much sharing exists. A PSS-style credit assumes a shared page's cost splits between its mappers, but cgroups implement no such split: the kernel charges a memory area in full to whichever cgroup instantiated it, and for an area used from different cgroups *"to which cgroup the area will be charged is in-deterministic"* [12]. A limit set below an instance's full resident footprint, assuming half of some shared region is billed to a neighbour, can therefore fail in either direction: the kernel may charge the *entire* region to that one instance and OOM-kill it, while the neighbour runs with headroom nobody budgeted for. This is consistent with Kubernetes requiring request and limit to be equal under its strictest QoS class [11].

One channel our measurement deliberately excludes deserves an explicit answer, because recent work builds a placement algorithm on it. Cheng et al. [3] report that the host buffer/cache footprint of five co-located containers of one function stays flat instead of growing fivefold, and place workflow functions to maximize that effect. Their quantity is the page cache of image-layer files, including content a process reads rather than maps, which §2 excludes from ours. The two results are compatible: content is cached once per inode whether or not anyone maps it, and only the mapped subset can enter a page-table intersection. It does not, however, rescue the cost-effectiveness argument. Page cache filled by reads is clean, reclaimable, and charged by cgroup v2 to whichever cgroup touched it first [12], so it fails as a cost-effectiveness lever twice: a Kubernetes request or OpenWhisk invoker slot is set in advance, not measured from what the kernel cached, and the kernel evicts the cache under pressure, so it is no committed claim on memory at all. Saving it lowers host memory pressure, but admits no additional container.

The stakes rise sharply for serverless inference, where a model server's resident memory is dominated not by a language runtime but by model weights: hundreds of megabytes to tens of gigabytes and byte-identical across every instance serving it. Zero-copy formats such as safetensors, or the GGUF files `llama.cpp` maps, enter the address space through

`mmap` and fall inside the quantity measured here; loaders that read weights into anonymous memory land in the page-cache channel instead. Either way the inode condition decides whether two co-located instances pay once or twice, so the mechanism of §5.1 applies unchanged – over objects three orders of magnitude larger, often delivered through mounted volumes rather than image layers, which changes which rule governs the inode. That holds only for weights resident in host RAM, though: once copied into GPU memory, each instance holds an independent device allocation outside the host page cache, a separate sharing problem this mechanism cannot reach. Measuring the host-RAM case is the obvious next step.

7 Threats to Validity

Our measurement counts only pages both instances map. Content a container reads without mapping – for Keycloak, the ~144 MiB of JARs – is still deduplicated in the page cache when the inode matches, a channel reported as substantial for small function images [3]. Our figures are therefore a lower bound, and the gap is application-dependent: a runtime mapping most of what it reads would show little difference, one reading most of it a large one. The measurements are strictly pairwise; host-level saving across more than two instances is bounded by one cached copy. The page-cache figures of §5.2–5.3 come from one exemplary set on an idle node; characterizing that channel across versions and under memory pressure is future work.

Our measurements come from one application family, a JVM service whose shareable memory sits in a few memory-mapped native libraries and a class-data-sharing archive. A different runtime profile – a statically-linked binary, or one mapping more of its own files – could realize a different fraction; the mechanism is not JVM-specific, but the magnitude is. The cross-version experiment rests on three pairs, adequate given the 0.02 MiB spread between same-build controls. Results come from Kubernetes, not a commercial FaaS environment; generalization rests on the kernel and OCI mechanism, both orchestrator-independent.

8 Conclusion

Crediting assumed memory sharing between co-located containers raises resource utilization, but the credit has never been measured precisely for a specific pair. Using pairwise page-frame intersection, we measured it for a JVM-based service in two situations: concurrent instances of one version, and tenant-pinned versions on a host. Sharing is small in the first case and collapses by an order of magnitude in the second, driven by how snapshots are identified, not by the application. A sharing discount should not be assumed from layer similarity alone, especially once co-located versions are built independently.

Acknowledgments

This research is partially funded by the Internal Funds KU Leuven. An AI assistant (Claude, Anthropic) was used while preparing this manuscript: for discussion and verification of the measurement methodology and of related work, for drafting and revising prose under the authors' direction, and for assembling the LaTeX scaffold. All experimental results, data, and conclusions are the authors' own, and AI-assisted text was reviewed and edited by the authors, who take full responsibility for the submitted content.

References

[1] Emad Heydari Beni, Eddy Truyen, Bert Lagaisse, Wouter Joosen, and Jordy Dieltjens. 2021. Reducing cold starts during elastic scaling of containers in kubernetes. In *Proceedings of the 36th Annual ACM Symposium on Applied Computing* (SAC '21). Association for Computing Machinery, Virtual Event, Republic of Korea, 60–68. ISBN: 9781450381048. doi:10.1145/3412841.3441887.

[2] José Bravo Ferreira, Marco Cello, and Jesús Omana Iglesias. 2017. More sharing, more benefits? a study of library sharing in container-based infrastructures. In *European Conference on Parallel Processing*. Springer, 358–371.

[3] Dazhao Cheng, Kai Yan, Xinquan Cai, Yili Gong, and Chuang Hu. 2024. SLO-aware function placement for serverless workflows with layer-wise memory sharing. *IEEE Transactions on Parallel and Distributed Systems*, 35, 6, 1074–1091. doi:10.1109/TPDS.2024.3391858.

[4] Jonathan Corbet. 2007. ELC: how much memory are applications really using? https://lwn.net/Articles/230975/. LWN.net; accessed: 2026-09-30. (2007).

[5] Grafana Labs. [n. d.] K6 official GitHub repository. https://github.com/grafana/k6. Accessed: 2026-05-01. ().

[6] Michael Kerrisk. [n. d.] Inode(7) — Linux manual page. https://man7.org/linux/man-pages/man7/inode.7.html. Accessed: 2026-05-01. The Linux man-pages project, ().

[7] Alexander Larsson. 2024. Composefs and containers. Talk at FOSDEM 2024. Accessed: 2026-10-01. (2024). doi:10.5446/71831.

[8] Wolfgang Mauerer. 2008. *Professional Linux Kernel Architecture*. Wiley Publishing, Inc., Indianapolis, IN. ISBN: 978-0-470-34343-2.

[9] Open Container Initiative. 2025. OCI image format specification. https://github.com/opencontainers/image-spec. Accessed: 2026-05-30. (2025).

[10] Divyanshu Saxena, Tao Ji, Arjun Singhvi, Junaid Khalid, and Aditya Akella. 2022. Memory deduplication for serverless computing with medes. In *Proceedings of the Seventeenth European Conference on Computer Systems* (EuroSys '22). Association for Computing Machinery, New York, NY, USA, 714–729. doi:10.1145/3492321.3524272.

[11] The Kubernetes Authors. 2026. Kubernetes quality of service classes. https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/. Accessed: 2026-05-29. (May 2026).

[12] The Linux Kernel Authors. [n. d.] Control group v2. https://docs.kernel.org/admin-guide/cgroup-v2.html. Accessed: 2026-05-29. ().

[13] The Linux Kernel Authors. [n. d.] Overlay filesystem. https://docs.kernel.org/filesystems/overlayfs.html. Accessed: 2026-05-30. ().

[14] Eddy Truyen, Bert Lagaisse, Wouter Joosen, Arnout Hoebreckx, and Cédric De Dycker. 2020. Flexible migration in blue-green deployments within a fixed cost. In *Proceedings of the 2020 6th International Workshop on Container Technologies and Container Clouds*, 13–18.